

Semester 6

Sem 5 / Sem 6

Watermarked study resource

Generated from local SRM dataset

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J DEC 2023.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

THE HELPERS

[Page 2]

THE HELPERS

[Page 3]

THE HELPERS

[Page 4]

THE HELPERS

[Page 5]

THE HELPERS

[Page 6]

THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J JAN 2024.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

THE HELPERS

[Page 2]

THE HELPERS

[Page 3]

THE HELPERS

[Page 4]

THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J JULY 2025.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

[Page 3]
THE HELPERS

[Page 4]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J MAY 2022.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J MAY 2023.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J MAY 2024.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

[Page 3]
THE HELPERS

[Page 4]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J MAY 2025.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

[Page 3]
THE HELPERS

[Page 4]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J NOV 2022.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC304J NOV 2024.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

[Page 3]
THE HELPERS

[Page 4]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/18CSC362J NOV 2023.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
THE HELPERS

[Page 2]
THE HELPERS

[Page 3]
THE HELPERS

[Page 4]
THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/IMPORTANT QUESTIONS.pdf =====
CATEGORY: PYQs
STATUS: ok

[Page 1]
List of Topics in Unit 1:
Phases of compiler
Cousins
Compiler Construction Tools
Role of Lexical Analyser
Input Buffering
Problems:
Direct conversion of RE to DFA
DFA
NFA to DFA

Epsilon NFA to DFA
RE to ENFA(Thompson)
Minimised DFA

List of Topics in Unit 2:
Role of parser
CFG
Leftmost, rightmost derivation
Parse trees
Ambiguity
Recursive Descent Parsing
Elimination of Left Recursion
Left Factoring
First and Follow
Predictive Parsing Table
Predictive parsing
LL(1) grammar

Students FYI:
Important part to know about CD FT1:
Unit 1

[Page 2]

1. phases of compiler
2. Compiler construction Tool
3. Min if DFA (Direct method) etc
4. Thompson method

Unit 2:

1. Predictive parser
2. Left factoring
3. Elimination of ambiguity

UNIT 3 – 4 Mark Questions

Leading and Trailing.

Lookahead and GOTO function.

LR(0), LR(1), Difference between LR(0) and LR(1).

UNIT 4 – 4 Mark Questions

Quadruple representation, Triples and Indirect Triples.

Target Code Generation.

Theory:-

Issues in Code Generation.

Target Machine, Input and Output.

Runtime Storage.

10 Mark Questions

SLR parsing.

LALR parsing.

CLR (Canonical LR) parsing.
Operator Precedence Parsing.
Shift reduce
S-attributed definitions.
Inherited attributes.

[Page 3]

PART B (8 Mark Questions) UNIT 1

- Phases of Compiler – 4 times
- DFA Diagram – 3 times
- Working of Compiler Phases – 3 times
- Role of Lexical Analyser – 2 times
- Input Buffering – 2 times
- Compiler Construction Tools – 2 times
- LEX Program

UNIT 2

- First and Follow – 6 times
- LL(1) Parser – 3 times
- Ambiguous Grammar – 2 times
- Derivation and Parse Tree – 2 times
- Leading and Trailing
- Operator Precedence Relation Table
- Assignment Statements using SDT
- Define CFG
- Parse Tree
- Top Down Parsing

UNIT 3

- Shift Reduce Parsing – 5 times
- LR(0), Closure of LR(0), GOTO of LR(0), SLR – 4 times
- LR(1), Closure of LR(1), GOTO of LR(1), LALR/CLR – 3 times
- LR Parsing – 2 times
- Bottom Up Parsing
- Operator Precedence Relation Table

UNIT 4

- Intermediate Code Generation – 3 times
- Types of Three Address Statements – 3 times
- Quadruples, Triples, Indirect Triples – 2 times
- Infix to Postfix

[Page 4]

- Code Optimisation Techniques

- Syntax Tree
- Simple Code Generator Algorithm
- Issues in Design of Code Generator
- Boolean Expression (Numerical Representation) using SDT
- Boolean Expression – Flow of Control using SDT

UNIT 5

- Peephole Optimisation – 4 times
- DAG – 4 times
- Global Data Flow Analysis – 2 times
- Principal Sources of Optimisation – 2 times
- Transformation on Basic Blocks – 2 times
- Basic Blocks and Flow Graphs

PART C (15 Mark Questions)

- RE to DFA – 4 times
- Quadruples, Triples and Indirect Triples – 2 times
- Backpatching – 2 times
- Derivation and Parse Tree – 2 times
- Code Generator Algorithm – 2 times
- First and Follow
- Runtime Environment
- Storage Organisation and Activation Records
- Infix to Postfix
- Syntax Tree
- Input Buffering
- LL(1) Parser
- Top Down Parsing
- Operator Precedence Parsing Table
- Boolean Expression (Numerical Representation) using SDT
- Storage Allocation Strategies
- Transformation on Basic Blocks

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/PYQ May 2024.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

THE HELPERS

[Page 2]

THE HELPERS

[Page 3]

THE HELPERS

[Page 4]

THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/PYQ May 2025.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

THE HELPERS

[Page 2]

THE HELPERS

[Page 3]

THE HELPERS

[Page 4]

THE HELPERS

===== SOURCE_FILE: sem 5 sem6/Semester 6/Compiler Design/PYQs/QUESTION BANK.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

SRMIST, KATTANKULATHUR School of Computing QUESTION BANK 18CSC304J- COMPILER DESIGN PART-A1. What is a translator? 2. Differentiate interpreters and compilers. 3. What is an assembler? 4. What are the phases of a compiler? 5. What is bootstrapping a compiler? 6. What are the two parts of compilation? Give a Diagram. 7. Give the parse tree for the statement $a := b + c * 60$. 8. What is the use of scanner generator? 9. What do you mean by syntax and semantics of a language? 10. Give some four compiler-generator tools. 11. Mention the cousins of the compiler. 12. Define a token. 13. What is a preprocessor? 14. What is a symbol table? 15. Define regular expressions. 16. Write down the Algebraic laws obeyed by Regular expressions. 17. Define a finite state automation. 18. Draw the NFA for a^*b^* . 19. Draw the NFA for $R = ((\epsilon \mid a) b^*)^*$. 20. Give the Procedure for ϵ -Closure. 21. What is a transition diagram? Give example. 22. What is a recognizer? 23. What is CFG? Give example. 24. What are the capabilities of CFG? 25. Describe language denoted by the following regular expression: $0(0/1)^*0$. 26. Describe language denoted by the following regular expression: $(0/1)^*0(0/1)(0/1)$. 27. Describe language denoted by the following regular expression: $0^*a0^*a0^*a0^*$. 28. Write the regular definition for the following language: "All strings of digits with no repeated digits". 29. Give an example for ambiguous grammar and justify it. 30. What is a parse tree? 31. What are the types of Grammars? 32. What is a Linear Grammar? Give ex.

[Page 2]

33. What do you mean by BNF? Give Ex. 34. Consider the grammar: $S \rightarrow (L) / aL \rightarrow L, S / S$ What are the terminals, non-terminals and start symbols? 35. Differentiate NFA and DFA. 36. What type of Grammar is the following, Define it. $S \rightarrow Aa \mid Bb \mid aA \rightarrow aA \mid cB \rightarrow bB \mid c$ 37. Given the grammar $S \rightarrow aSbS \mid bSaS \mid \epsilon$ What is the problem in this grammar? 38. What are the types of derivation? 39. What is right-most and left-most derivation? Give example. 40. What is left-sentential form and right-sentential form? 41. Minimise the given transition table: states a b A B C B B D C B C D B E E B C Initial State=A Final State=E 42. Write regular definition for a number which can have fraction or exponent part as optional. 43. For the following grammar: list $\rightarrow$ list+digit / list-digit / digit digit $\rightarrow$ 0/1/...../9 Derive the sentence (9+5-4) using left-most derivation. 44. What is a Lexical analyzer? 45. Distinguish tokens, patterns and lexemes. 46. What are the lexical-error recovery actions? 47. What is the need for input buffering? 48. Explain prefix and suffix of the string with example. 49. What is a transition diagram? Give example? 50. Draw the transition diagram for identifiers. 51. What is a parser? 52. What is the need for separating the parser from scanner? 53. What are the problems in Top down parsing? 54. What is ambiguity? Give Ex. 55. Consider the Grammar $S \rightarrow (L) \mid aL \rightarrow L, S \mid S$ 56. Construct the left most and Right most derivation for the following sentence (i) (a, a) ii) (a, ((a, a) , (a, a)))

[Page 3]

57. What is Left recursion? Write the rules to eliminate left recursion. 58. Consider the grammar $S \rightarrow aSbS \mid bSaS \mid \epsilon$ Show this grammar is ambiguous by constructing parse tree for the sentence abab. 59. Eliminate the left recursion for the following grammar $S \rightarrow (L) / aL \rightarrow L, S / S$ 60. Eliminate Left Recursion in the following Grammar. $A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$ 61. What is Left factoring? How do you eliminate it? 62. What is parsing table? 63. What is the use of LEX? 64. What is a handle? 65. What is handle pruning? 66. What do you mean by Precedence and Associativity? 67. Compare Shift-Reduce Parsing and Operator-Precedence Parsing 68. What are Precedence Matrices and Precedence Functions? 69. What is the use of Boolean Matrices? 70. Translate the arithmetic expression $a * -(b+c)$ into a syntax tree. 71. Write the FIRST and FOLLOW rules 72. Write the LEADING and TRAILING rules 73. What are LR(0) items? 74. When is a grammar called SLR(1)? 75. What is the advantage of using LALR parsing over SLR parsing? 76. Define the CLOSURE and GOTO procedures 77. Define a Quadruple. How it is different from Triples. 78. Convert the exp into three address code & Quadruple. $S = (a+b)/(c-d)*(e+f)$ 79. Write the prefix & postfix exp for the following $A = (20+(-5)*6+12)$ 80. What are Synthesized & Inherited Attributes? 81. Give example for inherited attributes. 82. Write the semantic actions for generating a Syntax tree 83. What are the procedures used for Syntax tree generation? 84. Draw the Syntax tree and DAG for $Y = C + D * T$; $C = C + D$; $X = C + D$; 85. Convert the following expression into Postfix and Triples $Z := (X-Y+K) / (A + B) * C$ 86. Convert the given exp into three address code a or b and not c 87. What do you mean by Backpatching, why we are going for backpatching. 88. Draw the syntax tree for the following exp $a := b * -c + b * -c$ 89. Write the postfix notation for the following exp

[Page 4]

$-(a+b)*(c+d)+(a+b+c)$. 90. Translate the exp $(a+b)*(c+d)+(a+b+c)$ into Quadruple b) Triples c) Indirect triples. 91. What is an S-attributed definition? 92. What is a L-attributed definition? 93. What are the issues in Code Generation 94. What do you mean by Peep-hole optimization? 95.

Construct DAG for the following exp $(a+b)-(e-(c+d))$.96. Write the mnemonic code for $(a+b)-(e-(c+d))$.97. Construct the DAG for the following basic block $d:=b*c$ $e:=a+b$ $b:=b*c$ $a:=e-d$ 98.What is Dangling Reference?99. What are the three storage allocation strategies?100. What do you mean by binding of a name?101. What are the components of an Activation record?102. What are the various parameter passing techniques?103. Give example for Call by reference104. Give example for Call by name105. What are Static & Stack allocations?106. What is the need for code optimization?107.What are the loop optimization techniques?108. Explain about common sub-expression elimination 109. What is the use of Dominator property in flow graphs?110. What are Basic Blocks? How to identify them? -----
----- PART-B1. Discuss the compiler construction tools.2. Discuss about the cousins of the compiler.3. Explain the phases of the compiler with diagram. Write down the output of each phase for the expression $a := b + c * 50$.4. Write notes on: (i) Regular expressions. (ii) Context Free Grammar.5. Show that the following grammar is ambiguous $S \rightarrow aSbS/bSaS/e$ 6. For a sentence belonging to the above grammar:a) Draw the parse tree.

[Page 5]

b) Construct the left-most and right-most derivations of "abab".7. Construct a minimal DFA for $(a/b)^*(a/b)$ and write the algorithm for minimizing a DFA.8. Explain the process of constructing an NFA from the regular expression.9. Find the NFA for the expression $(a/b)^*abb$.10. Construct the DFA for $(a/b)^*abb$ and minimise the DFA.11. Convert the Regular expression to DFA using Thompson's Construction $R = (a \mid b)^* a (a \mid b)$ 12. How would you construct a DFA directly from a regular expression? 13. Write the appropriate rules and algorithm wherever necessary.14. a. Discuss the Language for specifying Lexical analyzer b. Explain the design of a Lexical analyzer generator15. Explain the Thomson's construction technique in detail. Construct the NFA for the expression $d(a/b)^*a$.16. Explain Non recursive predictive parsing technique17. Explain shift-reduce parsing with stack implementation.18. Explain Shift Reduce Parsing and Perform the stack implementation to show that the string not (true or false) is parsed using the grammar given below
 $Bexpr \rightarrow Bexpr \text{ or } Bterm \mid Bterm$
 $Bterm \rightarrow Bterm \text{ and } Bfactor \mid Bfactor$
 $Bfactor \rightarrow \text{not } Bfactor \mid (Bexpr) \mid \text{true} \mid \text{false}$ 19. Explain the operator precedence parsing algorithm with example20. Explain the algorithm for constructing precedence functions with suitable example21. Construct SLR parsing table for $S \rightarrow AS \mid \epsilon$, $A \rightarrow SA \mid \epsilon$ 22.a. Check whether the following grammar is a LL(1) grammar
 $S \rightarrow iEtS \mid iEtSeS \mid aE \rightarrow b$ b. Explain the first and follow procedures to construct a predictive parser.23. Find (i) FIRST and FOLLOW(ii) LEADING and TRAILING for the grammer in Q.2224. Explain the method of constructing Parsing table and error recovery in predictive parsing.25. Construct predictive parser for the following grammar
 $S \rightarrow (L) \mid aL \rightarrow L,S \mid S$ 26. Check whether the following grammar is SLR(1) or not. Explain with reasons.
 $S \rightarrow L = RS \rightarrow RL \rightarrow *RL \rightarrow idR \rightarrow L$ 27. Construct LR(0) items for the grammar in question 26.

[Page 6]

28. a. Construct the LR(0) items for the Grammar given below (8)
 $E \rightarrow E + T \mid T$ $T \rightarrow TF \mid F$ $F \rightarrow F * \mid a \mid b$ b. Construct a SLR parsing table for the above 29. Explain the LALR parsing method with example.30. Show that the Grammar is LL(1) but not SLR(1)
 $S \rightarrow AaAb \mid BbBa$ $A \rightarrow \epsilon$ $B \rightarrow \epsilon$ 31. Write short notes on: a) Left factoring b) Handle pruning c) Left recursion & backtracking32. Explain LALR parsing, justify how it is efficient over SLR parsing.33.

Explain the various Intermediate codes with suitable ex.34. Explain the process of translating assignment statements35. Describe the process of translating CASE statements.36. Explain the method of constructing Syntax trees for expressions with suitable procedures.37. Discuss the Inherited attributes and Dependency graphs with an example38. Discuss Back Patching with example39. With a grammar explain the translation of Boolean expressions using Backpatching method. b.Explain the backpatching process for the statement given below $((a > b) \text{ or } ((c > d) \text{ or } (e < f)))$ 40. Describe the process of translating Boolean Expressions.41. Explain how flow of control statements are translated.42. Explain the various parameter passing mechanisms of a high level language with examples.43. Discuss how access to non-local names is done in storage allocation strategies?44. Explain in detail about Run Time Storage Allocations.45. Explain in detail about Local & loop Optimization techniques46. Explain Frequency reduction and Strength reduction techniques with examples47. Explain in detail about Peep-Hole Optimization with examples.48. Discuss the Data Flow Analysis with example49. Explain the issues in the design of code generator and design a simple Code generator.50. Write the algorithm for constructing the DAG . Explain how to optimize using DAG.-----

===== SOURCE_FILE: sem 5 sem6/Semester 6/Data Science/PYQs/Jul 2024.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)

lOMoARcPSD|37189447

[Page 2]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)

lOMoARcPSD|37189447

[Page 3]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)

lOMoARcPSD|37189447

[Page 4]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)

lOMoARcPSD|37189447

===== SOURCE_FILE: sem 5 sem6/Semester 6/Data Science/PYQs/May 2024.pdf =====

CATEGORY: PYQs

STATUS: ok

[Page 1]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)
10MoARcPSD|37189447

[Page 2]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)
10MoARcPSD|37189447

[Page 3]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)
10MoARcPSD|37189447

[Page 4]

Downloaded by AMAN DEEP SINGH (RA2311003011710) (as0300@srmist.edu.in)
10MoARcPSD|37189447